Chapter 2

Path Types

interval 上の位置を表す離散的な可算無限個の名前 $i, j, k, \ldots$ が与えられているとする。この名前の集合上の自由 de Morgan 代数（*free de Morgan algebra*）、すなわち名前を生成元とし、Definition 10 の等式（と束の公理）以外の関係を一切課さずに式として生成した de Morgan 代数を、$\mathbb{I}$ と定める。

Definition 11 (The Interval $\mathbb{I}$)**.** $\mathbb{I}$ の元は次の文法で与えられる（i は名前）：

$$r, s \quad ::= \quad 0 \mid 1 \mid i \mid \mathtt{1\text{-}}r \mid r \sqcap s \mid r \sqcup s$$

$\mathbb{I}$ は最大元 1・最小元 0 をもつ有界分配束であり、対合（involution）$\mathtt{1\text{-}}r$ は以下を満たす：

$$\begin{array}{ll} \mathtt{1\text{-}}0 = 1 & \mathtt{1\text{-}}(r \sqcup s) = \mathtt{1\text{-}}r \sqcap \mathtt{1\text{-}}s \\ \mathtt{1\text{-}}1 = 0 & \mathtt{1\text{-}}(r \sqcap s) = \mathtt{1\text{-}}r \sqcup \mathtt{1\text{-}}s \\ \mathtt{1\text{-}}(\mathtt{1\text{-}}r) = r & \end{array}$$

$\mathbb{I}$ の等値性は決定可能であり、$\mathbb{I}$ は生成元 i, $\mathtt{1\text{-}}i$ 上の自由分配束としても記述できる（決定可能性はこの記述による：各元は「生成元の交わりの結び」という標準形をもち、標準形の比較に帰着する。後に $\mathbb{F}$ について同種の議論を Lemma 20 で詳述する）。$\mathbb{I}$ の元は、各名前の指す位置（区間 $[0,1]$ の元）を定めるごとに $[0,1]$ の元を**値**として定める式とみなせる：$r \sqcap s$ の値は $\min(r, s)$、$r \sqcup s$ の値は $\max(r, s)$ である[*1]。ただし一般に $\mathtt{1\text{-}}r \sqcap r \neq 0$ かつ $\mathtt{1\text{-}}r \sqcup r \neq 1$ であることに注意。

2.1 Syntax and Inference Rules

文脈は名前宣言で拡張できるようになる：

$$\Gamma, \Delta \quad ::= \quad \ldots \mid \Gamma, i : \mathbb{I}$$

これに対応する文脈規則は次で与えられる（判断 Γ *context* は、Γ が well-formed な文脈であることを表す。右辺をもつ判断と紛れないよう、この判断には $\vdash$ を用いない）：

$$\frac{\Gamma \ \mathit{context}}{\Gamma, i : \mathbb{I} \ \mathit{context}} \quad \mathit{where}\ i \notin \mathit{dom}(\Gamma).$$

[*1] 原論文 (Cohen et al., 2018) は $\mathbb{I}$ の束演算を $r \wedge s$, $r \vee s$ と書くが、本書では face lattice $\mathbb{F}$ (Section 3.1) の演算 $\wedge$, $\vee$ と区別するため $\sqcap$, $\sqcup$ を用いる。

判断 $\Gamma \vdash r : \mathbb{I}$ は、Γ *context* が成り立ち、かつ $\mathbb{I}$ の元 r が Γ で宣言された名前のみに依存することを意味する。判断 $\Gamma \vdash r = s : \mathbb{I}$ は r と s が $\mathbb{I}$ の元として等しいことを意味する（ただし $\mathbb{I}$ の判断的等値性は、Section 3.1 で制限付き文脈を導入した際に再定義される）。とくに、宣言された名前自身はつねに $\Gamma, i : \mathbb{I}, \Gamma' \vdash i : \mathbb{I}$ を満たす。これは対象変数に対する変数規則（VAR）の名前版として機能するが、i は対象変数ではなく**名前**である—この身分の違いと、名前を「sort $\mathbb{I}$ をもつ変項」とみなす正確な定式化については Remark 14 で論じる。後の導出（たとえば Section 2.2 の ap が (Path$E$) に $i : \mathbb{I}$ を渡す箇所）では、この事実を断りなく用いる。

基本的な依存型理論の構文への拡張は次のとおりである：

$$t, u, A, B \quad ::= \quad \ldots \mid \mathrm{Path}\, A\, t\, u \mid \langle i \rangle\, t \mid t\, r \qquad (\textit{path type})$$

path 抽象 $\langle i \rangle\, t$ は t の中の名前 i を束縛し、path 適用 $t\, r$ は項 t を元 $r : \mathbb{I}$ に適用する。項の代入操作は、名前 i に $\mathbb{I}$ の元 r を代入する操作 $[r/i]$ へ拡張される[*2]。とくに $[0/i]$ と $[1/i]$ は、n 次元立方体の面（face）を取る操作に対応する。

Definition 12 (Inference Rules for Path Types)**.**

$$\frac{\Gamma \vdash A : \mathrm{type} \quad \Gamma \vdash u_0 : A \quad \Gamma \vdash u_1 : A}{\Gamma \vdash \mathrm{Path}\, A\, u_0\, u_1 : \mathrm{type}}\ (\mathrm{Path}F)$$

$$\frac{\Gamma \vdash A : \mathrm{type} \quad \Gamma, i : \mathbb{I} \vdash t : A}{\Gamma \vdash \langle i \rangle\, t : \mathrm{Path}\, A\, t[0/i]\, t[1/i]}\ (\mathrm{Path}I) \qquad \frac{\Gamma \vdash t : \mathrm{Path}\, A\, u_0\, u_1 \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash t\, r : A}\ (\mathrm{Path}E)$$

なお、この box と次の box を通じて、メタ変数は役割で書き分けている：u_0・u_1 は Γ 上の元（**点**＝端点）であり、t（および (Pathη) の u）は i 方向の広がりをもつもの—(Path$I$)・(Pathβ) の前提では $\Gamma, i : \mathbb{I}$ で型付けされる**線**、(Path$E$) 以下では path 型の元—である。とくに (Path$I$) では、前提の線 t の 2 つの面 $t[0/i]$・$t[1/i]$ が、結論の型において端点の位置に立つ。

Definition 13 (Computation Rules for Path Types)**.**

$$\frac{\Gamma \vdash A : \mathrm{type} \quad \Gamma, i : \mathbb{I} \vdash t : A \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash (\langle i \rangle\, t)\, r = t[r/i] : A}\ (\mathrm{Path}\beta) \qquad \frac{\Gamma, i : \mathbb{I} \vdash t\, i = u\, i : A}{\Gamma \vdash t = u : \mathrm{Path}\, A\, u_0\, u_1}\ (\mathrm{Path}\eta)$$

$$\frac{\Gamma \vdash t : \mathrm{Path}\, A\, u_0\, u_1}{\Gamma \vdash t\, 0 =_\partial u_0 : A}\ (\partial) \qquad \frac{\Gamma \vdash t : \mathrm{Path}\, A\, u_0\, u_1}{\Gamma \vdash t\, 1 =_\partial u_1 : A}\ (\partial)$$

なお (Pathη) では t, u が適切に型付けされているという前提を省略している（原論文どおり）。端点の 2 規則 (∂) の等号に付した添字の意味—これらが β 簡約とは別種の、向

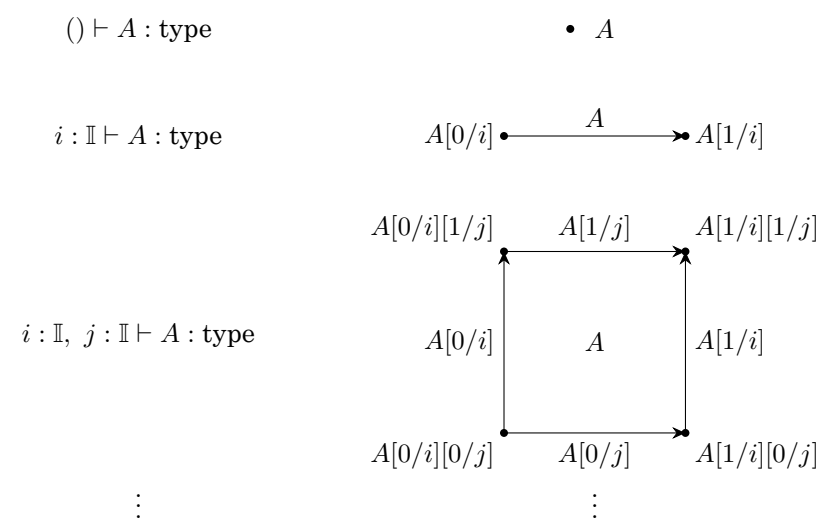
[*2] 本書の代入記法は原論文のそれと異なる。Cohen et al. (2018) は、名前 i を r で置き換える代入を (i/r) と書き、とくに $(i/0)$, $(i/1)$ を単に $(i0)$, $(i1)$ と略記している。本書では、項の代入記法 $[L/x]$ に合わせて、これらをそれぞれ $[r/i]$, $[0/i]$, $[1/i]$ と書く。原論文と代入の向きが逆になっている点に注意されたい。本書の $[L/x]$ が「x に L を代入する」を表すのに対し、原論文の (i/r) は「i に r を代入する」を表す。また通常の代入は preterm への preterm の代入であるが、本書ではこれを 2 つの方向へ拡張して用いる。第一に、代入されるものが preterm ではなく $\mathbb{I}$ の元である。第二に、代入を受ける側が preterm のほかに、face formula（Section 3.1、たとえば $\varphi[r/i]$）や $\mathbb{I}$ の元自身（たとえば $r[0/i]$）にも及ぶ。いずれの場合も、束縛された名前の下では代入が起こらないこと、および必要に応じて α-変換を行うことは、通常の項の代入と同様である。

き付けられない等値性であること—は Section 3.6 で論じる。反射律の証明に対応する項 1_a を

$$1_a \stackrel{def}{\equiv} \langle i \rangle\, a : \mathrm{Path}\, A\, a\, a$$

と定義する。

直観的には、名前を含む文脈における判断は立方体を表す。ここで注意したいのは、このことが **2 つのレベル**で起こることである。第一に、名前を含む文脈で型付けされた**項** $i : \mathbb{I} \vdash p : A$（$A$ は i を含まない）は、型 A の**中の**線である。第二に、名前を含む文脈における**型** $i : \mathbb{I} \vdash A : \mathrm{type}$ は、**型そのものの線**である。原論文の図は後者で例示される：

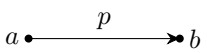


ここで $A[0/i][0/j] = A[0/j][0/i]$ が成り立つことに注意。

図の $A[0/i]$ は、それ自身が（i を含まない）1 つの型である。すなわちこの線分は、どれか 1 つの型の**中の**図形ではなく、**型たちの並び**であり、面を取る操作 $[0/i], [1/i]$ は族から成員の型を取り出す。これが「何の**中の**」線なのか—型の宇宙の中の path—を体系の中で言えるようになるのは、sort $\mathbf{type}_i$ どうしの path を扱う Chapter 5 においてであり、当面は「i に依存する型」という判断レベルの概念として扱えば足りる（後述の Remark 14 も参照）。型の線は、依存 path 型（Section 2.3）と移送（Chapter 3）で主役になる。

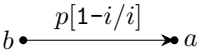
このように、名前は、ある interval 上を走る変数であるが、転じて、その interval 自体—立方体の 1 つの軸—を指すこともある。後者の用法を「i の方向（direction）」と呼ぶ。解析幾何で、座標変数 x に因んでそれが走る軸を「x 軸」と呼ぶのと同じ習わしである。代入 $[0/i], [1/i]$ が意味をもつのは、前者の用法（interval 上を走る変数）に対してである[*3]。代入 $[j/i]$ は次元の名前替えに、$[1\text{-}i/i]$ は path の反転に対応する。

項のレベルでも同じ図が描ける：$i : \mathbb{I} \vdash p : A$ が $p[0/i] = a$, $p[1/i] = b$ を満たすとき、p は方向 i の線分



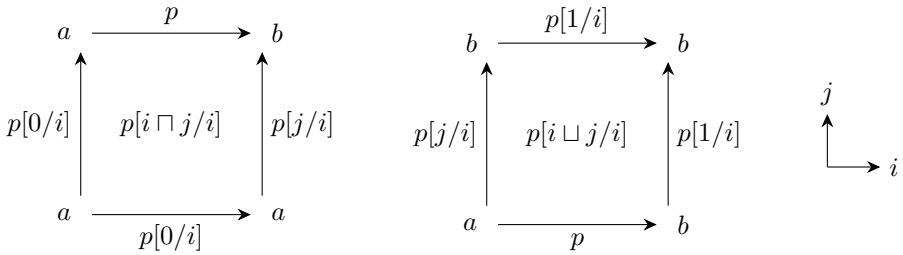
[*3] 原論文 (Cohen et al., 2018) は、名前を最初から「direction を表す」ものとして導入する。しかし、名前が方向（軸）そのものを表すのだとすると、そこに interval の端点を代入する操作の意味が通らない。そこで本書では、interval 上を走る変数としての用法を基本とし、「方向」はそこからの転用として区別する。

とみなせ、その反転は次のようになる：



名前替えの代入 $[j/i]$ には単射性の制約がないことに注意しよう。したがって、立方体の「対角線」を取る代入も可能である：A が $i : \mathbb{I}, j : \mathbb{I}$ に依存する正方形なら、$A[j/i]$ はその対角線（方向 j の線分）を与える。

代入 $[i \sqcap j/i]$ と $[i \sqcup j/i]$ は *connection* と呼ばれる特別な退化（degeneracy）に対応する。connection $p[i \sqcap j/i]$ と $p[i \sqcup j/i]$ は次の正方形として描ける：



たとえば左の正方形の右辺は

$$p[i \sqcap j/i][1/i] = p[1 \sqcap j/i] = p[j/i]$$

と計算され、下辺と左辺は退化している（定値 path $p[0/i]$）。このように connection は、低次元の立方体から高次元の立方体を構築する便利な手段を与える。型のレベルでも同様であり、A が i に依存する線型（line type）なら、$A[i \sqcap j/i]$ は上の $p[i \sqcap j/i]$ と同じ形の正方形（の内部）を与える。

Remark 14 (名前の二義性について)**.** 本節の冒頭で述べた、名前の「interval 上を走る変数／転じて軸」という二義性は、本書の理解のつまずきの種になりやすいので、立ち止まって整理しておく。解析幾何の x, y にも同じ曖昧さはあるが、構文の厳密な運用をこそ旨とする型理論の文脈では、この曖昧さは際立つからである。

まず、二つの読みは同格ではない。軸の読みが可能なのは**名前だけ**である。たとえば $i \sqcap j$ や $1\text{-}i$ は、$\mathbb{I}$ の元として名前の指す位置から値を定めはするが、「$i \sqcap j$ の軸」なるものは存在しない。立方体の軸は、文脈に宣言された名前 $i, j, \ldots$ と 1 対 1 に対応するのであって、$\mathbb{I}$ の一般の元と対応するのではない（解析幾何でも、x 軸・y 軸はあるが「$\min(x, y)$ 軸」はない）。

次に、代入 $[r/i]$ の i の位置に立てるのも名前だけである。connection のように複合的な元を名前へ代入する $[i \sqcap j/i]$ は許されるが、逆に複合的な式への「代入」—たとえば $[i/i \sqcap j]$ や $[i/1\text{-}j]$—は書けない。その根拠は、名前が**変項**（variable）だという点にある。代入とは変項への代入であり、これは項の代入 $[N/x]$ が変項 x に対してだけ定義されるのと同じ理由である。$[i/1\text{-}j]$ に相当するものが欲しければ、それは代入ではなく「方程式 $1\text{-}j = i$ を解く」操作であって、体系には存在しない。

実のところ、正確な定式化は次のようになるはずである：名前とは sort $\mathbb{I}$ をもつ変項であり、Definition 11 の BNF は、判断 $\Gamma \vdash r : \mathbb{I}$ の形成規則（0, 1 は $\mathbb{I}$ の元、文脈に宣言された名前は $\mathbb{I}$ の元、$\mathbb{I}$ の元は $1\text{-}(\cdot)$・$\sqcap$・$\sqcup$ で閉じる）を圧縮した略記である。本書が名前宣言 $i : \mathbb{I}$ と判断 $\Gamma \vdash r : \mathbb{I}$ を用いるのは、この定式化の一部を実際に採っているということであり、$\mathbb{I}$ の等式の側も、判断 $\Gamma \vdash r = s : \mathbb{I}$ として扱われる（Section 3.1 で制限文脈とともに再定義するのがそれである）。それでも $\mathbb{I}$ を型理論の完全な一部とはしない—$\mathbb{I}$ は型ではなく、$\mathbb{I}$ 上の関数型も $\mathbb{I}$ の除去規則も存在しない—のは、提示を軽くするた

めであり、原論文も同じ簡略化を採っている。

最後に、暗黙のままになっているものを指摘しておく。最たるものは立方体そのものである。「n 次元立方体」は構文上の対象ではない：体系に立方体を表す式は存在せず、「判断が n 個の interval 変項を含む」という、**判断のレベルの性質**として現れるだけである。本節の図で正方形を「描けた」のは、項とその文脈の**組**を眺めたからであって、どの単独の式もあの正方形を表してはいない。次元とは、式の属性ではなく判断の属性なのである。ただし、これは上の簡略化の代価**ではない**ことに注意しよう。仮に $\mathbb{I}$ を正式な型として体系に加えたとしても、立方体が「文脈に $\mathbb{I}$ の変項を n 個含む判断」として現れるという事情は変わらない。立方体の暗黙性は、次元を文脈の項目（仮定）として扱うという体系の設計そのものに由来する。なお、本節で導入した Path 型と path 抽象 $\langle i \rangle\, t$ は、この判断レベルの線を、両端の情報とともに項のレベルへ内在化する装置とみなすことができる（正方形についても、依存 path 型（Section 2.3）の反復により同様の内在化が得られる）。

2.2 Examples

等値性を path 型で表現すると、等号型では等号型の除去によって証明される多くの標準的操作を、新しい仕方で直接定義できる。以下では、そのような構成を派生規則の形で掲げる。すなわち以下の各規則は、体系への新しい規則の追加ではなく、前提のもとで結論に掲げた項が実際にその型をもつという、証明つきの主張である。最初の 2 つの規則では、前提に現れる型の well-formedness（$\Gamma \vdash A : \mathrm{type}$ など）も前提として明示する。以降の例では、これらは暗黙の前提として省略することがある。

■等しい元の像は等しい（ap）「等しい元の像は等しい」ことは、次の規則として導出できる：

$$\frac{\Gamma \vdash A : \mathrm{type} \qquad \Gamma \vdash B : \mathrm{type} \qquad \Gamma \vdash f : A \to B \qquad \Gamma \vdash a : A \qquad \Gamma \vdash b : A \qquad \Gamma \vdash p : \mathrm{Path}\, A\, a\, b}{\Gamma \vdash \langle i \rangle\, f\,(p\, i) : \mathrm{Path}\, B\,(f\, a)\,(f\, b)}$$

実際、(Path$I$) によりこの項は $\mathrm{Path}\, B\,(f\,(p\,0))\,(f\,(p\,1))$ の元であり、端点の ∂-等値性 $p\,0 =_\partial a$, $p\,1 =_\partial b$ により、この型は結論の $\mathrm{Path}\, B\,(f\, a)\,(f\, b)$ に判断的に等しい。この操作は homotopy type theory で ap（action on paths）と呼ばれるものにあたり、本書でも後の証明の記述でこの名を用いる。また、この操作は、等号型を帰納族として定義した場合には判断的には成り立たない判断的等値性をいくつか満たす。

■関数外延性　等号型では導出できなかった（Theorem 1）関数外延性（function extensionality）が、path 型では証明できる：

$$\frac{\Gamma \vdash A : \mathrm{type} \qquad \Gamma, x : A \vdash B : \mathrm{type} \qquad \Gamma \vdash f : (x : A) \to B \qquad \Gamma \vdash g : (x : A) \to B \qquad \Gamma \vdash p : (x : A) \to \mathrm{Path}\, B\,(f\, x)\,(g\, x)}{\Gamma \vdash \langle i \rangle\, \lambda x{:}A.\; p\, x\, i : \mathrm{Path}\,((x : A) \to B)\, f\, g}$$

これが正しいことを確かめるには、この項が**正しい面（face）をもつ**ことを確認すればよい。この言い方の意味を明確にしておこう。path 抽象の本体 $t \overset{def}{\equiv} \lambda x{:}A.\; p\, x\, i$ は名前 i を自由に含む項、すなわち $(x : A) \to B$ の中の線であり、その**面**とは、代入で得られる両端 $t[0/i]$ と $t[1/i]$ のことである（Section 2.1 で「面を取る操作」と呼んだ $[0/i]$, $[1/i]$ による）。正しい面をもてば十分である理由は次のとおり。(Path$I$) がこの項に与える型は $\mathrm{Path}\,((x : A) \to B)\, t[0/i]\, t[1/i]$ であるから、面が意図した端点に判

断的に等しいこと—$t[0/i] = f$ かつ $t[1/i] = g$—さえ成り立てば、この型は結論の型 $\mathrm{Path}\,((x : A) \to B)\,f\,g$ に判断的に等しく、conversion により結論の型付けが得られる。すなわち型付けの義務は、この 2 つの判断的等値性に帰着する。

0 の側の面を確認しよう：

$$(\langle i \rangle\, \lambda x{:}A.\ p\,x\,i)\,0 =_\beta \lambda x{:}A.\ p\,x\,0 =_\partial \lambda x{:}A.\ f\,x =_\eta f$$

この連鎖が $t[0/i] = f$ の証明（判断的等値性の導出）とみなせるのは、各等号がいずれも本書の等値性規則の適用だからである。第一の等号は ($\mathrm{Path}\beta$) であり、左辺を代入形 $t[0/i] = \lambda x{:}A.\ p\,x\,0$ へ簡約する（連鎖を path 適用の形から書き出したのはこのためで、面の計算と同じことである）。第二の等号は、$p\,x : \mathrm{Path}\,B\,(f\,x)\,(g\,x)$ に対する端点の ∂-等値性 $p\,x\,0 =_\partial f\,x$（と、それを λ の下で使う判断的等値性の合同性）による。第三の等号は Π 型の η 規則による（本書の基礎体系は Π の η を陽に含む。原論文も同様である）。以下本書では、判断的等値性の検算の連鎖では、各等号に由来を添えて種別を明示する：∂-等値性による段には $=_\partial$ を、その他の段にも $=_\beta \cdot =_\eta \cdot =_{\mathrm{def}}$（定義の展開）のように添える（複合する段は $=_{\mathrm{def},\beta}$ のように併記する）。なお、path の**合成**による連鎖（Theorem 74 の証明など）の等号は判断的等値性ではないから、この添字の対象外である。1 の側も同様にして、$t[1/i] = g$ が確かめられる。

■積型の path の分解　等号型のもとでは、$A \times B$ の元どうしの等式が成分ごとの等式に分解すること

$$(a, b) =_{A\times B} (a', b') \quad \longleftrightarrow \quad (a =_A a') \times (b =_B b')$$

は証明を要する命題であり、両方向とも J（`idpeel`）で構成される。Path 型では、同じ内容が判断的等値性のレベルで—すなわち定義的に—成立する。まず両方向の写像を、導出可能な規則として掲げる：

$$\frac{\Gamma \vdash p : \mathrm{Path}\,(A \times B)\,(a, b)\,(a', b')}{\Gamma \vdash \langle i \rangle\, \pi_1(p\,i) : \mathrm{Path}\,A\,a\,a'} \qquad \frac{\Gamma \vdash q : \mathrm{Path}\,A\,a\,a' \quad \Gamma \vdash r : \mathrm{Path}\,B\,b\,b'}{\Gamma \vdash \langle i \rangle\, (q\,i, r\,i) : \mathrm{Path}\,(A \times B)\,(a, b)\,(a', b')}$$

第 2 射影の側 $\langle i \rangle\, \pi_2(p\,i)$ も同様である。型付けの義務は、funext のときと同じく面の検算に帰着する。たとえば左の規則の 0 の側は

$$\begin{array}{rcll} (\langle i \rangle\, \pi_1(p\,i))\,0 & =_\beta & \pi_1(p\,0) & (\mathrm{Path}\beta) \\ & =_\partial & \pi_1((a, b)) & \text{端点の } (\partial) \text{ 規則と合同性} \\ & =_\beta & a & \text{積の } \beta \end{array}$$

であり、残りの面も同様に確かめられる。略記 $\pi_1^*(p) \overset{def}{\equiv} \langle i \rangle\, \pi_1(p\,i)$（$\pi_2^*$ も同様）、$\mathrm{pair}^*(q, r) \overset{def}{\equiv} \langle i \rangle\, (q\,i, r\,i)$ を導入すると、両者の往復は**判断的に**恒等である：

$$\begin{array}{rcll} \pi_1^*(\mathrm{pair}^*(q, r)) & =_{\mathrm{def}} & \langle i \rangle\, \pi_1((\langle j \rangle\, (q\,j, r\,j))\,i) & \text{定義の展開} \\ & =_\beta & \langle i \rangle\, \pi_1((q\,i, r\,i)) & (\mathrm{Path}\beta) \\ & =_\beta & \langle i \rangle\, q\,i & \text{積の } \beta \\ & =_\eta & q & (\mathrm{Path}\eta) \end{array}$$

$$\begin{array}{rcll} \mathrm{pair}^*(\pi_1^*(p), \pi_2^*(p)) & =_{\mathrm{def},\beta} & \langle i \rangle\, (\pi_1(p\,i), \pi_2(p\,i)) & \text{定義の展開と } (\mathrm{Path}\beta) \\ & =_\eta & \langle i \rangle\, p\,i & \text{積の } \eta \text{ (surjective pairing)} \\ & =_\eta & p & (\mathrm{Path}\eta) \end{array}$$

第 2 の連鎖で用いた積の η（surjective pairing；$(\pi_1(u), \pi_2(u)) = u$）は、Π の η と同じく本書の基礎体系に陽に含まれる（原論文も同様である）。すなわち 2 つの写像は互いに逆であり、合成は判断的に恒等である（定義的同型）。注目すべきは、この構成が—本節

のここまでの例と同じく—次章で導入する合成の操作（Kan 構造）を一切使っていないことである。効いているのは、Path 型の元が区間からの関数であって $p\,i$ という適用ができること、そして端点の等値性が**判断的**であることだけである。等号型では、p の中身に触れる手段が J しかないため、この対応は命題として証明され、往復も propositional にしか成り立たない。

■**singleton の可縮性**　singleton の可縮性、すなわち $(x : A) \times \mathrm{Path}\, A\, a\, x$ の任意の元が $(a, 1_a)$ と等しいことも正当化できる：

$$\frac{\Gamma \vdash p : \mathrm{Path}\, A\, a\, b}{\Gamma \vdash \langle i \rangle\, (p\, i, \langle j \rangle\, p\, (i \sqcap j)) : \mathrm{Path}\, ((x : A) \times \mathrm{Path}\, A\, a\, x)\, (a, 1_a)\, (b, p)}$$

Exercise 15. 連結 $p[i \sqcap j/i]$ が Section 2.1 の図の正方形になること、すなわちその 4 つの面が p, $p[0/i]$（定値）, $p[j/i]$, $p[0/i]$（定値）であることを、$\mathbb{I}$ の等式（$0 \sqcap j = 0$, $1 \sqcap j = j$）を用いて確かめよ。また $p[i \sqcup j/i]$ について同じ計算を行え。

Exercise 16. path の反転を $p^{-1} \stackrel{def}{\equiv} \langle i \rangle\, p[1\text{-}i/i]$ と定義する。$(p^{-1})^{-1} = p$ が**判断的**に成り立つことを、$\mathbb{I}$ の対合の性質 $1\text{-}(1\text{-}i) = i$ から示せ。（p^{-1} と p の合成が定値 path に等しいことは、これに対して判断的には成り立たず、次章の hcomp を要する。）

2.3 Dependent Path Types

Definition 12 の $\mathrm{Path}\, A\, u_0\, u_1$ では、型 A は方向 i に依存しない。しかし、型の線 $\Gamma, i : \mathbb{I} \vdash A : \mathrm{type}$ に沿って、$A[0/i]$ の元と $A[1/i]$ の元を結びたい場面がある。これを表すのが依存 *path* 型（dependent path type）である。

Definition 17 (Dependent Path Types)**.**

$$\frac{\Gamma, i : \mathbb{I} \vdash A : \mathrm{type} \quad \Gamma \vdash u_0 : A[0/i] \quad \Gamma \vdash u_1 : A[1/i]}{\Gamma \vdash \mathrm{PathP}^i\, A\, u_0\, u_1 : \mathrm{type}}\ (\mathrm{PathP}F)$$

$$\frac{\Gamma, i : \mathbb{I} \vdash A : \mathrm{type} \quad \Gamma, i : \mathbb{I} \vdash t : A}{\Gamma \vdash \langle i \rangle\, t : \mathrm{PathP}^i\, A\, t[0/i]\, t[1/i]}\ (\mathrm{PathP}I) \qquad \frac{\Gamma \vdash t : \mathrm{PathP}^i\, A\, u_0\, u_1 \quad \Gamma \vdash r : \mathbb{I}}{\Gamma \vdash t\, r : A[r/i]}\ (\mathrm{PathP}E)$$

β・η・端点の規則は Definition 13 と同じ形である。

A が i に依存しない場合が Definition 12 の Path であり、

$$\mathrm{Path}\, A\, u_0\, u_1 = \mathrm{PathP}^i\, A\, u_0\, u_1 \qquad (A \text{ が } i \text{ を含まないとき})$$

が成り立つ。以下、Path は PathP の特別な場合として扱う。

依存 path 型には 2 つの用途がある。第一に、Section 4.1 以降で高次帰納型の除去を扱う際に必要となる。高次帰納型 D の除去とは、motive と呼ばれる型の族 $P : D \to \mathrm{type}$ について、すべての $x : D$ に対する $P\, x$ の要素を構成することであり、そのためには構成子ごとに「その構成子の上での P の要素」を与えればよい。このとき path 構成子の上で与えるべき要素は、両端点の上の要素どうしを結ぶ path であるが、この path が各時点 i で通過する型は、構成子の path 上の点に P を適用した型であり、i とともに変化する。したがって、この path の型は PathP でなければ書けない[*4]。第二に、Section 3.4

[*4] 円周 S^1（Section 4.2）で先取りして述べる。S^1 は point 構成子 `base` と path 構成子 `loop` をも

の transp と組み合わせると、依存 path と「移送したうえでの通常の path」との対応

$$\mathrm{PathP}^i\, A\, u_0\, u_1 \quad \text{と} \quad \mathrm{Path}\, A[1/i]\, (\mathtt{transp}^i\, A\, 0_{\mathbb{I}}\, u_0)\, u_1$$

が得られる（両者は同値である）。これは homotopy type theory における `apd` と「`transport` を経由した等式」の関係にあたる。この同値の構成は Exercise 48 で扱う。

2.3.1 The Challenge of Justifying the Elimination Rule

Path 型・PathP 型が等号型の代替となるためには、等号型の各規則に対応するものが path 型の側で成り立たなければならない。形成規則 (=F) に対応するのは (Path$F$) ((PathPF)) である。導入規則 (=I)—`refl`—に対応するものも、すでに導出可能である：

$$\frac{\Gamma \vdash A : \mathrm{type} \quad \Gamma \vdash t : A}{\Gamma \vdash \langle i \rangle\, t : \mathrm{PathP}^i\, A\, t\, t}$$

実際、A も t も i を含まないから、weakening と (PathPI) により $\langle i \rangle\, t$ は $\mathrm{PathP}^i\, A\, t[0/i]\, t[1/i]$ の型をもち、その両面はいずれも t 自身である。この項は Section 2.1 で定義した 1_t にほかならない（A が i を含まないから $\mathrm{PathP}^i\, A\, t\, t = \mathrm{Path}\, A\, t\, t$ でもある）。

問題は除去規則である。Path 型・PathP 型は、形成・導入・除去・β・η という型構成子としての一式を既に備えてはいる。しかし、ここでの除去規則 (Path$E$), (PathPE) は path を $\mathbb{I}$ の元に適用するだけの規則であり、そこから取り出せるのは path 上の点にすぎない。等号型の除去規則 `idpeel`（J 規則）が果たしていた役割—$p : \mathrm{Path}\, A\, a\, b$ と motive から「等しいものを置き換える」推論を引き出し、path を**消費**する—に対応する規則は、まだ何も与えられていない。このままでは、Path 型を等号型の代替として用いることはできない。

そこで課題となるのが、J に対応する除去規則の**正当化**である。ここで正当化とは、その規則を公理として体系に追加するのではなく、その型をもつ項を構成することで**導出可能**にすることを指す。ただし、先に断っておくと、この仕方で回復されるのは型付けの部分だけである：`idpeel` の計算規則（β 規則）に対応する等式は、導出された除去規則については判断的等値性としては成り立たず、Path の項として成り立つにとどまる (Section 3.7 で論じる)。

この構成のために体系へ追加されるのが、合成（composition）と総称される操作である。これは cubical set の Kan 構造に由来する用語で、「立方体の、一部の面が欠けた境界（開いた箱）が与えられたとき、欠けた面を埋める」操作を指す。位相的な path の合成 (Definition 7) の形式版 $p \cdot q$ を最も単純な場合として含むことが、名称の由来である。本書ではこれを 2 つの操作—型が方向に沿って変化しない homogeneous composition と、型の線に沿って元を運ぶ移送—に分けて導入する。これが次章の主題である。

History and Further Reading

本章の枠組み—interval 上の名前による文脈の拡張と path 型—は Cohen et al. (2018) の §3 にもとづく（本書の記法との差異は本文の脚注で個別に断った）。名前を用いた形式

ち、motive $P : S^1 \to \mathrm{type}$ に沿った除去では、`base` の上の要素 $\bar{c} : P(\mathtt{base})$ と、`loop` の上の要素 $\bar{l} : \mathrm{PathP}^i\, P(\mathtt{loop}\, i)\, \bar{c}\, \bar{c}$ の 2 つを与えればよい。後者は $\bar{c}$ から $\bar{c}$ 自身への path だが、i が動くにつれて通過する型 $P(\mathtt{loop}\, i)$ 自体が変化する（P が定値関数でない限り、どの単一の型 A の中にも収まらない）ので、その型は Path では書けない。

化は nominal な λ-計算の系譜に連なり、その経緯と、cubical set モデルへ至る歴史的背景については Chapter 1 と Chapter 3 の History を参照。

Bibliography

Altenkirch, T., C. McBride, and W. Swierstra. (2007) "Observational equality, now!", In Proceedings of *the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.

Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) "Syntax and models of Cartesian cubical type theory", *Mathematical Structures in Computer Science* **31**(4), pp.424–468.

Angiuli, C., R. Harper, and T. Wilson. (2017) "Computational higher-dimensional type theory", In Proceedings of *the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.

Awodey, S. and M. A. Warren. (2009) "Homotopy theoretic models of identity types", *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.

Bekki, D. (2014) "Representing Anaphora with Dependent Types", In Proceedings of *Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France), LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.

Bekki, D. and K. Mineshima. (2017) "Context-Passing and Underspecification in Dependent Type Semantics", In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.

Bezem, M., T. Coquand, and S. Huber. (2014) "A Model of Type Theory in Cubical Sets", In Proceedings of *19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

Cavallo, E. and R. Harper. (2019) "Higher Inductive Types in Cubical Computational Type Theory", *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.

Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) "Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom", In Proceedings of *21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.

Coquand, T., S. Huber, and A. Mörtberg. (2018) "On Higher Inductive Types in Cubical Type Theory", In Proceedings of *LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.

Hofmann, M. (1995) "Extensional concepts in intensional type theory", Ph.D. thesis, University of Edinburgh.

Hofmann, M. and T. Streicher. (1998) "The groupoid interpretation of type theory",

In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.

Huber, S. (2019) "Canonicity for Cubical Type Theory", *Journal of Automated Reasoning* **63**(2), pp.173–210.

Kapulkin, K. and P. L. Lumsdaine. (2021) "The simplicial model of Univalent Foundations (after Voevodsky)", *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.

Lumsdaine, P. L. and M. Shulman. (2020) "Semantics of higher inductive types", *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.

Martin-Löf, P. (1975) "An intuitionistic theory of types", In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium '73*. Amsterdam, North-Holland, pp.73–118.

Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).

Mörtberg, A. (2021) "Cubical methods in homotopy type theory and univalent foundations", *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.

Nordström, B., K. Petersson, and J. Smith. (1990) *Programming in Martin-Löf's Type Theory*. Oxford University Press.

Nordström, B., K. Petersson, and J. M. Smith. (2000) "Martin-Löf's Type Theory", In: *Handbook of Logic in Computer Science, Vol 5*. Oxford University Press.

Orton, I. and A. M. Pitts. (2016) "Axioms for Modelling Cubical Type Theory in a Topos", In Proceedings of *25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: Logical Methods in Computer Science 14(4:23), 2018.

Sterling, J. and C. Angiuli. (2021) "Normalization for Cubical Type Theory", In Proceedings of *36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.

Streicher, T. (1993) "Investigations into Intensional Type Theory", Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.

The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. `https://homotopytypetheory.org/book`.

Vezzosi, A., A. Mörtberg, and A. Abel. (2019) "Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types", *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.